

# ORBIT-MVP Project Documentation

## Table of Contents

1. [Project Overview](#)
2. [Why This Tech Stack?](#)
3. [Architecture and Workflow](#)
4. [Component Breakdown](#)
5. [AWS Infrastructure and Integrations](#)
6. [Deployment Lifecycle](#)
7. [Security and IAM](#)
8. [Cost Management](#)
9. [Limitations and Future Improvements](#)
10. [50 Viva Questions & Answers](#)

## 1. Project Overview

ORBIT-MVP is a lightweight, self-hosted Platform as a Service (PaaS) engine designed to provide zero-downtime rolling deployments of applications from a Git repository to a Kubernetes cluster running on AWS (EKS) or a local mock cluster (kind). It condenses an entire cloud delivery pipeline (CI/CD) into a localized Go-based control plane.

## 2. Why This Tech Stack?

- **Golang:** High concurrency, minimal footprint, robust standard library, excellent AWS SDK and Kubernetes client (`client-go`).
- **AWS CodeBuild:** Fully managed build service. Cheaper and more scalable than running Jenkins nodes.
- **Amazon ECR:** Secure container registry integrated deeply with AWS IAM.
- **Kubernetes (EKS / kind):** Industry standard container orchestration, providing self-healing, scaling, and zero-downtime rolling updates.

## 3. Architecture and Workflow

```
graph TD
    A[Developer Git Push] --> B[ORBIT Web Dashboard]
    B -->|Triggers Build API| C[Go Backend (main.go)]
    C -->|Invokes AWS SDK| D[AWS CodeBuild]
    D -->|Pulls source, builds Docker image| E[AWS ECR (Registry)]
    E -->|Reports status| C
    C -->|kubectl apply / client-go| F[Kubernetes Cluster]
    F -->|Pulls Image| E
    E -->|Exposes App| G[LoadBalancer / Port-Forward]
```

## 4. Component Breakdown

- **main.go:** The HTTP server. Serves the embedded HTML frontend and handles the REST API routes for deployment status and triggering builds.
- **deploy.go:** The orchestration logic. It interacts with AWS SDKs to trigger CodeBuild, parses status, and constructs/applies Kubernetes manifests (Deployments & Services).
- **web/index.html:** A static, responsive frontend that polls the Go backend for real-time deployment logs and statuses.

## 5. AWS Infrastructure and Integrations

- **EKS:** Managed Kubernetes cluster.
- **CodeBuild:** Used to execute Docker builds based on a dynamically generated buildspec.
- **ECR:** Used to store the built images securely.

## 6. Deployment Lifecycle

1. User provides Git URL in Dashboard.
2. Go backend creates an AWS CodeBuild project (or mocks it locally).
3. CodeBuild pulls the source code, builds the `Dockerfile`, and pushes to ECR.
4. Once built, Go backend generates a Kubernetes Deployment manifest referencing the new ECR image tag.
5. The manifest is applied to the cluster, performing a rolling update.
6. A Kubernetes Service of type `LoadBalancer` (or `ClusterIP` for local) is configured to expose the app.

## 7. Security and IAM

- **IAM Roles:** The EKS worker nodes need IAM roles to pull images from ECR. CodeBuild needs IAM roles to push to ECR.
- **Secrets:** Avoiding hardcoded AWS credentials by relying on the default AWS credential chain (`~/.aws/credentials`).

## 8. Cost Management

- A teardown script (`scripts/destroy-eks.ps1`) is provided to completely wipe the EKS cluster, returning compute costs to \$0/hour.

## 9. Limitations and Future Improvements

### Implemented:

- Local fallback (kind).
- AWS CodeBuild / EKS integration.
- Real-time frontend polling.

### Future:

- Proper Webhook integration (instead of manual API triggers).
- HTTPS/TLS termination using Let's Encrypt.
- Multi-tenant namespaces.

## 10. 50 Viva Questions & Answers

1. **What is ORBIT-MVP?** A simplified PaaS engine to deploy Git repos to Kubernetes.
2. **Why use Go for the backend?** It compiles to a single binary, is extremely fast, and has excellent libraries for Kubernetes and AWS.
3. **What is the role of `main.go`?** It runs the web server and API endpoints.
4. **What is the role of `deploy.go`?** Orchestrates CodeBuild and Kubernetes deployments.
5. **How does the frontend get updates?** It polls the backend API endpoints.
6. **What is a fallback mode?** If AWS is unavailable, it builds locally and deploys to `kind`.
7. **What is `kind`?** Kubernetes IN Docker, used for local testing.
8. **Why AWS CodeBuild?** It handles docker builds serverlessly.
9. **Why AWS ECR?** Securely stores docker images in AWS.
10. **How are zero-downtime deployments achieved?** Through Kubernetes RollingUpdates.
11. **What is an Application Load Balancer?** It routes external HTTP traffic into the Kubernetes cluster.
12. **How does ORBIT authenticate with AWS?** Using the default credential chain / SDK config.
13. **What happens during a build phase?** Source code is downloaded, docker image is built, and pushed to ECR.
14. **How are Kubernetes resources defined in ORBIT?** They are generated dynamically via Go structs and applied using `client-go` or `kubect1`.
15. **What is a Dockerfile?** Instructions for building a Docker container image.
16. **Why use a rolling update strategy?** To ensure the app remains available while new pods are started.
17. **What is a Kubernetes Pod?** The smallest deployable unit in Kubernetes, containing one or more containers.
18. **What is a Kubernetes Deployment?** A higher-level abstraction that manages Pods and replica scaling.
19. **What is a Kubernetes Service?** An abstraction to expose an application running on a set of Pods.
20. **What is the difference between ClusterIP and LoadBalancer?** ClusterIP is internal only; LoadBalancer provisions an external cloud load balancer.
21. **How is cost managed in this project?** By providing scripts to easily destroy expensive resources like EKS.
22. **What language is the frontend?** Vanilla HTML/CSS/JS.
23. **How does the system handle concurrent deployments?** The Go backend handles requests asynchronously via goroutines.
24. **What is the primary benefit of a PaaS?** Developers can push code without worrying about servers or infrastructure.
25. **Is the database managed by ORBIT?** No, currently it only deploys stateless applications.
26. **What happens if a deployment fails?** Kubernetes will pause the rollout and the previous version remains active.
27. **Can ORBIT deploy Python applications?** Yes, as long as they have a Dockerfile (e.g., Flask).
28. **Can ORBIT deploy Node.js applications?** Yes, using a Dockerfile.
29. **How is the Kubernetes manifest generated?** Using Go templates or `k8s.io/api` libraries.
30. **What is `kubect1`?** The command-line tool for interacting with a Kubernetes cluster.
31. **What is `eksctl`?** A tool used to easily create and manage EKS clusters.
32. **Why delete the cluster when not in use?** EKS control planes and EC2 nodes incur hourly charges.
33. **What is an IAM Role?** An AWS identity with permission policies that determine what the identity can and cannot do in AWS.
34. **Does ORBIT support automatic Github webhooks?** It is planned for the future, but currently relies on manual triggers via the dashboard.
35. **What is `go mod tidy` used for?** To clean up and organize Go dependencies in `go.mod`.
36. **How does the backend store state?** In memory for this MVP version.
37. **What happens if the Go server restarts?** In-memory state of running deployments is lost.
38. **How would you make the state persistent?** By adding a database like PostgreSQL or Redis.
39. **What is the role of `.dockerignore`?** To prevent unnecessary files from being copied into the Docker image, speeding up builds.
40. **How are environment variables passed to the app?** They can be injected into the Kubernetes Deployment manifest.
41. **What is a Namespace in Kubernetes?** A logical isolation boundary within a cluster.
42. **Which namespace does ORBIT deploy to?** `orbit-apps`.
43. **Why use `docker system prune`?** To clear out unused docker images and free up disk space.
44. **What is the biggest limitation of this MVP?** Lack of persistent storage and authentication.
45. **How would you secure the ORBIT dashboard?** By adding OAuth2, JWT, or Basic Auth.
46. **What is the port for the Go server?** Default is usually 8080.
47. **Can you scale the application deployed by ORBIT?** Yes, by changing the replica count in the generated Kubernetes Deployment.
48. **What is AWS SDK?** A software development kit provided by AWS to interact with their services programmatically.
49. **What is Continuous Integration?** Automating the building and testing of code every time a team member commits changes.
50. **What is Continuous Deployment?** Automatically deploying every change that passes automated tests to production.